Although you can get a reference to the current object through self, there's no equivalent keyword to get a reference to the current class. However, in most situations it's easy to keep track of the current class just by looking at the code. Here's how:

- At the top level of your program, the current class is Object, the class of main. (That's why, if you define a method at the top level, that method becomes an instance method of Object.)
- In a method, the current class is the class of the current object. (Try defining a method inside another method with def, and you'll see that the new method is defined on the class of self. This information is probably going to win you some Ruby trivia contest.)
- When you open a class with the class keyword (or a module with the module keyword), that class becomes the current class.

This last case is probably the only one that you care about in practice. Indeed, you use it all the time when you open a class with the class keyword, and define methods in the class with def. However, the class keyword has a limitation: it needs the name of a class. Unfortunately, in some situations you may not know the name of the class that you want to open. For example, think of a method that takes a class and adds a new instance method to it:

```
def add_method_to(a_class)
  # TODO: define method m() on a_class
end
```

How can you open the class if you don't know its name? You need some way other than the class keyword to change the current class. Enter the class_eval method.

class_eval()

Module#class_eval (also known by its alternate name, module_eval) evaluates a block in the context of an existing class:

class_definitions/class_eval.rb

```
def add_method_to(a_class)
  a_class.class_eval do
    def m; 'Hello!'; end
  end
end

add_method_to String
"abc".m   # => "Hello!"
```